

Report di Progetto – ISW2 Modulo 2

Software Testing su Apache Avro (fork lucaaa02)

[Nome Cognome] – [Matricola]

[Data]

Contents

1	Introduzione	1
1.1	Apache Avro	1
1.2	Classi selezionate	1
2	Infrastruttura di progetto	1
2.1	Build e gestione delle dipendenze	1
2.2	Integrazione continua e misurazione della copertura	1
2.3	Isolamento dei test del progetto	2
3	Test manuali (Category Partition)	2
3.1	La classe Resolver	2
3.2	Test manuali su Resolver	2
3.3	Copertura strutturale di resolver	3
3.4	La classe Utf8	3
3.5	Test manuali su Utf8	3
3.6	Scoperta: comportamento non intuitivo di setByteLength	4
3.7	Copertura strutturale di utf8	4
4	Test automatici	4
4.1	Test automatici su Resolver	4
4.2	Test automatici su Utf8	5
4.3	Integrazione tecnica di EvoSuite	6
5	Validazione della qualità dei test	6
5.1	Metriche di adeguatezza	7
5.2	Mutation testing con PIT	7
5.3	Stima della reliability	8
6	Refactoring e validazione incrociata	8
6.1	Refactoring di Resolver	8
6.2	Refactoring di Utf8	9
6.3	Sintesi	10
7	Conclusioni	10
A	Tabelle, figure e listati	i

1 Introduzione

L'elaborato ha come obiettivo la realizzazione di un'attività di verifica software su un fork di Apache Avro, applicata a due classi del progetto.

La verifica è stata condotta attraverso diverse metodologie di testing: test manuali basati sulla tecnica Category-Partition, test generati automaticamente tramite Randoop, test generati con l'ausilio di un LLM, e test guidati dalla copertura tramite EvoSuite.

Sono stati isolati i test relativi alle due classi in esame, escludendo dall'esecuzione la suite di test già presente nel progetto originale.

1.1 Apache Avro

Apache Avro è un framework di serializzazione dati, nato all'interno del progetto Hadoop. Fornisce un formato binario compatto ed efficiente per la codifica dei dati, un sistema di schemi (definiti in formato JSON) che descrivono la struttura dei dati serializzati, e un meccanismo di *schema evolution*, ovvero la possibilità di far comunicare tra loro versioni diverse dello stesso schema senza rompere la compatibilità. Quest'ultimo aspetto è direttamente rilevante per il progetto: una delle due classi analizzate (*Resolver*) implementa proprio la logica di risoluzione tra schema *writer* e schema *reader*.

1.2 Classi selezionate

La selezione delle classi è avvenuta applicando l'algoritmo di *class ranking* descritto nella consegna, basato sull'iniziale del cognome dello studente applicata alla classifica dei code smell rilevati da SonarCloud sul progetto Apache Avro.

Una prima classe candidata, è stata scartata per banalità, in accordo con l'indicazione della consegna di evitare classi il cui testing risulti banale o impraticabile, è stata selezionata la classe successiva.

Le classi definitivamente selezionate sono:

- `org.apache.avro.Resolver` (473 LOC, 25 code smell): classe di utilità che confronta, ricorsivamente, l'albero dello schema usato da chi ha *scritto* i dati (*writer*) con l'albero dello schema atteso da chi li deve *leggere* (*reader*). Per ogni coppia di nodi corrispondenti, determina quale azione è necessaria per adattare i dati scritti al formato atteso in lettura (nessuna azione, promozione di tipo, adattamento di record/enum/union, oppure segnalazione di incompatibilità), producendo un albero di oggetti `Resolver.Action`. È il meccanismo alla base della *schema evolution* di Avro, ovvero la capacità di far comunicare correttamente produttori e consumatori di dati che utilizzano versioni diverse dello stesso schema;
- `org.apache.avro.util.Utf8` (172 LOC, 8 code smell): classe mutabile che rappresenta una sequenza di testo codificata direttamente come array di byte UTF-8, anziché come `String` Java (rappresentata internamente in UTF-16). Implementa le interfacce `CharSequence`, `Comparable` ed `Externalizable`, fornendo metodi per impostare e leggere il contenuto, confrontare istanze e calcolarne l'hash. È utilizzata dal modello dati generico di Avro per rappresentare i campi di tipo stringa evitando le conversioni di codifica ripetute che sarebbero necessarie utilizzando `String`.

2 Infrastruttura di progetto

2.1 Build e gestione delle dipendenze

Il progetto Avro è organizzato come progetto multi-modulo Maven, con un file `pom.xml` principale che coordina i sotto-progetti, tra cui il modulo `lang/java/avro/`, dove abbiamo lavorato, che contiene le classi `Resolver` e `Utf8` selezionate per il testing.

È stato necessario l'utilizzo di tre versioni diverse del JDK: 11, 17 e 21. Il progetto Avro richiede infatti la compatibilità con JDK 11 per il codice compilato; inoltre, Randoop ed EvoSuite, generando codice sulla base della JVM su cui vengono eseguiti, devono essere lanciati anch'essi con JDK 11, per evitare di generare riferimenti a metodi non disponibili nella versione richiesta per la compilazione finale. Le versioni JDK 17 e 21 sono invece necessarie per compilare ed eseguire il progetto nelle altre fasi della build.

2.2 Integrazione continua e misurazione della copertura

È stata configurata un'integrazione continua tramite GitHub Actions: ad ogni push sul branch principale, il file di configurazione `isw2-ci.yml` esegue automaticamente una serie di step, tra cui la compilazione del progetto e l'esecuzione dei test su una macchina virtuale remota. Per la misurazione della copertura è stato utilizzato JaCoCo, analizzando le metriche di statement e branch coverage, mentre SonarCloud è stato integrato come supporto per l'analisi della qualità del codice.

L'integrazione continua si è rivelata utile per individuare problemi non visibili nell'ambiente di sviluppo locale: in particolare, un conflitto tra il JDK usato per la compilazione in CI (JDK 21) e il codice generato

da EvoSuite (eseguibile correttamente solo su JDK 11) si è manifestato soltanto durante l'esecuzione su GitHub Actions.

Sono stati inoltre utilizzati Spotless e Apache RAT per garantire la qualità del codice e la conformità alle licenze. Spotless applica automaticamente le regole di formattazione del progetto (indentazione, ordine degli import), mentre Apache RAT verifica che ogni file sorgente contenga l'header di licenza Apache 2.0, in linea con le politiche della Apache Software Foundation. Entrambi i controlli vengono eseguiti prima della fase di test, durante la compilazione: un file che non li rispetta blocca l'intera build, indipendentemente dal fatto che i suoi test vengano poi effettivamente eseguiti. Poiché i file di test generati automaticamente da Randoop ed EvoSuite non rispettano lo stile del progetto né includono l'header di licenza, è stato necessario configurare esclusioni esplicite per queste cartelle in entrambi gli strumenti.

2.3 Isolamento dei test del progetto

L'esecuzione dei test già presenti nel progetto è stata disabilitata, mantenendo attiva una suite di test composta unicamente dai test sviluppati per questo lavoro.

3 Test manuali (Category Partition)

I test manuali sono stati progettati e implementati secondo la tecnica Category-Partition, che consiste nell'identificare le categorie di comportamento osservabile di una classe, e nel definire per ciascuna categoria un insieme di valori rappresentativi da utilizzare come input per i test. L'approccio seguito è prevalentemente black-box.

Questa tecnica è stata applicata a entrambe le classi selezionate, individuando per ciascuna un insieme di categorie corrispondenti alle diverse configurazioni di comportamento osservabile. Le sezioni seguenti descrivono, per ciascuna classe, la struttura interna rilevante ai fini del testing, le categorie individuate e i risultati ottenuti.

3.1 La classe `Resolver`

La classe `Resolver` è una classe che si occupa di determinare come adattare i dati scritti in uno schema (*writer*) a uno schema di lettura (*reader*) potenzialmente diverso. Implementa un algoritmo in grado di confrontare ricorsivamente gli alberi degli schemi e produrre un albero di azioni necessarie per la conversione dei dati.

Il risultato prodotto è un'istanza della classe astratta `Resolver.Action`, il cui tipo concreto a runtime determina la specifica trasformazione da applicare: `Action` non viene mai istanziata direttamente, ma sempre tramite una delle sue sottoclassi. La **tabella 1** riassume quali sono i possibili tipi di azioni che la classe può restituire.

3.2 Test manuali su `Resolver`

Il metodo `resolve()` accetta due parametri principali: lo schema del *writer* e lo schema del *reader* (un terzo parametro opzionale, `GenericData`, interviene solo nel recupero di valori di default e conversioni logiche, e non introduce ulteriori categorie di comportamento rilevanti ai fini del testing). Il criterio di testing adottato si è basato sull'interazione tra le diverse combinazioni possibili di questi due parametri. Sono state individuate otto categorie di test, ciascuna corrispondente a una delle otto sottoclassi di `Resolver.Action` discusse in precedenza.

L'approccio adottato è prevalentemente black-box, dato che le regole di comportamento attese (quali promozioni numeriche sono ammesse, come si risolvono record, enum e union) derivano dalla specifica pubblica di schema evolution di Avro. Tuttavia, la *verifica* del risultato in ciascun test avviene tramite l'operatore `instanceof` su nomi di classi concrete interne (ad esempio `Resolver.DoNothing`, `Resolver.Promote`), informazione non derivabile dalla sola documentazione esterna: la progettazione dei casi di test è quindi black-box, mentre la loro implementazione richiede un elemento di conoscenza white-box del codice sorgente.

- **DoNothing**: verificata su tutti gli otto tipi primitivi di Avro, con schemi identici tra writer e reader.
- **Promote**: verificato che le promozioni numeriche valide (ad esempio `INT→LONG`, `FLOAT→DOUBLE`) producano l'azione corretta, e che le promozioni non valide generino invece un errore.
- **ErrorAction**: verificato che le incompatibilità irrisolvibili tra i due schemi generino correttamente un errore.
- **Container**: verificato che, per `ARRAY` e `MAP`, la risoluzione ricorsiva degli elementi interni funzioni correttamente.
- **EnumAdjust**: verificato che la rimappatura dei simboli tra writer e reader funzioni correttamente.
- **RecordAdjust**: verificato che la risoluzione campo per campo del record funzioni correttamente.

- **ReaderUnion**: verificato che la ricerca del ramo della union compatibile con il tipo scritto funzioni correttamente.
- **WriterUnion**: verificato che la risoluzione di ogni possibile ramo scritto dal writer rispetto al reader funzioni correttamente.

Per alcune categorie, in particolare `Promote`, i valori scelti per i test non sono stati selezionati arbitrariamente all'interno della categoria, ma rappresentano i confini tra comportamento valido e non valido: per ogni promozione numerica valida testata (ad esempio `INT→LONG`), è stata verificata anche la direzione opposta, non valida (`LONG→INT`), proprio in corrispondenza del limite tra le due categorie. La **tabella 2** riporta, per ciascuna categoria, i valori concreti utilizzati nella progettazione dei test.

In totale sono stati scritti ed eseguiti 32 test manuali, tutti passanti: non è emersa alcuna anomalia comportamentale in questa fase. La **tabella 6** riporta l'elenco completo dei 32 test, con l'input e l'assert atteso per ciascuno. I test sono collocati nel file `TestResolverCategoryPartition.java`, all'interno del package `org.apache.avro` del progetto.

3.3 Copertura strutturale di `resolver`

I dati di copertura sono stati misurati con JaCoCo, isolando l'esecuzione della sola suite di test manuali. La **tabella 7** riporta i valori di *statement coverage* (42.6%) e *branch coverage* (31.6%) ottenuti; la **fig. 1** mostra il dettaglio per metodo.

La copertura risultante è parziale: il `category-partition`, per sua natura, individua le categorie a partire dalle funzionalità e dal comportamento atteso della classe, non dall'obiettivo di massimizzare la copertura di ogni singola riga o diramazione del codice. Una copertura non completa non indica quindi una debolezza della suite, ma una caratteristica intrinseca della tecnica adottata: la relazione tra copertura strutturale e adeguatezza dei test manuali rispetto alle altre tecniche impiegate viene approfondita nel Capitolo 5, mentre i mutanti sopravvissuti nelle porzioni di codice non coperte sono discussi nel Capitolo 5.2.

3.4 La classe `Utf8`

La classe `Utf8` si occupa di rappresentare una sequenza di testo direttamente come byte codificati in UTF-8. In Java, il tipo `String` è rappresentato internamente come una sequenza di caratteri UTF-16: se un dato testuale dovesse essere sempre convertito da `String` a byte UTF-8 e viceversa, si introdurrebbe un costo computazionale ripetuto e spesso non necessario. La classe `Utf8` risolve questo problema rappresentando direttamente il testo come byte UTF-8, evitando conversioni superflue e convertendo in `String` solo quando effettivamente richiesto.

La classe implementa tre interfacce:

- `CharSequence`: consente di trattare l'istanza come se fosse testo, fornendo metodi quali `length()`, `charAt()` e `subSequence()`;
- `Comparable<Utf8>`: permette il confronto tra due istanze ai fini dell'ordinamento;
- `Externalizable`: fornisce un meccanismo di serializzazione a basso livello, tramite i metodi `writeExternal` e `readExternal`.

Gli attributi principali della classe sono: un array di byte (`bytes`), contenente la rappresentazione UTF-8 del testo; un intero (`length`), che indica quanti byte dell'array sono effettivamente utilizzati (l'array può essere più grande della lunghezza logica del contenuto, per consentire il riutilizzo della stessa area di memoria a fronte di modifiche successive); una cache opzionale della `String` corrispondente; una cache opzionale dell'hash calcolato.

La **tabella 4** elenca i metodi pubblici principali della classe.

3.5 Test manuali su `Utf8`

La classe `Utf8` è composta da più metodi indipendenti: sono state definite sei categorie di test, raggruppate per area funzionale correlata.

- **Costruzione**: verificata la corretta inizializzazione dell'istanza a partire da ciascuna delle modalità di creazione disponibili (costruttore vuoto, da stringa vuota, ASCII e Unicode, da stringa `null`, di copia, da array di byte vuoto e ASCII).
- **Uguaglianza e ordinamento**: verificato il comportamento di `equals()` e `compareTo()` rispetto alla stessa istanza, a contenuto identico, a contenuto diverso a parità di lunghezza, a lunghezza diversa, e nel confronto con `null` o con un oggetto di tipo diverso. Per questa categoria, l'approccio adottato non è stato puramente black-box: `equals()` e `hashCode()` implementano internamente due percorsi di calcolo distinti, uno ottimizzato per array esattamente dimensionati e uno tramite loop manuale, con soglia di attivazione a 7 byte. L'individuazione di questo confine ha richiesto un'ispezione mirata del codice sorgente, introducendo un elemento di conoscenza white-box a supporto della progettazione dei casi di test limite.

- **Gestione della lunghezza:** verificato il comportamento di `setByteLength()` in aumento, in diminuzione, a lunghezza invariata e nel caso limite di azzeramento.
- `CharSequence`: verificato il comportamento di `charAt()` (primo carattere, ultimo carattere, indice fuori intervallo) e di `subSequence()` (intervallo vuoto, valido, non valido).
- **Serializzazione:** verificato che una coppia di operazioni `writeExternal/readExternal` (round-trip) preservi correttamente il contenuto, su un'istanza vuota, su contenuto ASCII e su contenuto Unicode.
- `getBytesFor`: verificato il comportamento del metodo statico di utilità su stringa vuota, stringa ASCII, stringa Unicode e argomento `null`.

La **tabella 5** riporta l'elenco completo dei 35 test, con l'input/azione e l'assert atteso per ciascuno.

In totale sono stati scritti ed eseguiti 35 test manuali. La scoperta emersa dal test relativo alla gestione della lunghezza in aumento è discussa nel dettaglio nella sezione successiva.

3.6 Scoperta: comportamento non intuitivo di `setByteLength`

Il test T19, relativo alla gestione della lunghezza in aumento tramite il metodo `setByteLength()`, ha rivelato un comportamento non intuitivo nella classe `Utf8`. In particolare, quando si aumenta la lunghezza logica di un'istanza già popolata oltre la capacità fisica del buffer interno, ci si aspetterebbe che il contenuto originale venga preservato e che i nuovi byte aggiunti siano inizializzati a zero. Il test ha invece mostrato che, in questo caso, l'intero buffer viene riallocato e azzerato, perdendo il contenuto precedente.

La scoperta è avvenuta empiricamente e in modo puramente black-box: l'assert inizialmente scritto, basato sull'aspettativa naturale di preservazione del contenuto, falliva all'esecuzione. Solo in un secondo momento, ispezionando il codice sorgente, è emerso che questo comportamento è noto e documentato internamente dagli sviluppatori di Avro tramite un commento nel metodo (non riportato nella Javadoc pubblica): se il buffer deve crescere, viene riallocato e azzerato; se invece la nuova lunghezza rientra nella capacità già allocata, il puntatore di lunghezza si sposta semplicemente, lasciando intatto il contenuto precedente. Gli autori descrivono esplicitamente questa asimmetria come un compromesso deliberato a favore delle prestazioni, per evitare di azzerare il buffer anche quando non è strettamente necessario.

Questo comportamento, sebbene intenzionale, comporta un'implicazione di correttezza più sottile rispetto al solo azzeramento in fase di espansione: se un'istanza viene prima ridotta e poi nuovamente espansa entro la capacità già allocata, il buffer non viene azzerato, ed è possibile che restino byte residui (*stale*) risalenti al contenuto precedente alla riduzione. Un chiamante che assumesse che i byte aggiunti in espansione siano sempre azzerati potrebbe quindi osservare dati non pertinenti senza che ciò generi alcun errore visibile. Questo scenario specifico è stato oggetto di un test dedicato nella suite di mutation testing (Sezione 5.2).

Il test T19 è stato mantenuto, riformulato per verificare il comportamento realmente osservato: la scoperta dimostra il valore della metodologia black-box nel far emergere, in modo indipendente dalla conoscenza dell'implementazione, discrepanze tra il comportamento atteso da un utilizzatore della classe e il comportamento documentato internamente ma non esposto pubblicamente.

3.7 Copertura strutturale di `utf8`

I dati di copertura sono stati misurati con JaCoCo, isolando l'esecuzione della sola suite di test manuali. La **tabella 7** riporta i valori di *statement coverage* (66.7%) e *branch coverage* (43.2%) ottenuti; la **fig. 2** mostra il dettaglio per metodo.

Anche in questo caso la copertura risulta parziale, per lo stesso motivo discusso per `Resolver`: le categorie individuate rappresentano il comportamento funzionale atteso della classe, non un obiettivo di copertura strutturale completa.

4 Test automatici

Oltre ai test manuali basati su Category-Partition, sono stati applicati tre approcci di generazione automatica dei test, ciascuno basato su un principio diverso: il *random testing*, che genera input in modo casuale tramite reflection (Randoop); la generazione assistita da un modello di intelligenza artificiale (LLM), a partire da diverse formulazioni del prompt; la generazione *coverage-guided*, che ottimizza la ricerca di nuovi test verso la massimizzazione della copertura del codice (EvoSuite). Per ciascuna classe, i tre approcci sono stati applicati e confrontati tra loro, evidenziando punti di forza e limiti complementari.

4.1 Test automatici su `Resolver`

Randoop Randoop (versione 4.3.3) è uno strumento di *feedback-directed random testing*: genera sequenze di chiamate a metodi con input casuali, sfruttando la reflection Java. Lo strumento richiede di essere eseguito con lo stesso JDK usato per compilare il progetto (JDK 11): se lanciato con una JVM più recente, genera codice che utilizza API non disponibili nella versione richiesta per la compilazione finale (ad

esempio `isEmpty()` su `CharSequence`, introdotto in Java 15).

Con un budget di generazione di 60 secondi sono stati generati 3508 test, nessuno dei quali error-revealing. La **tabella 8** riporta i risultati numerici completi per entrambe le classi; le **fig. 3** e **fig. 6** mostrano il dettaglio per metodo. La copertura della sola suite Randoop, misurata isolatamente, risulta molto bassa su `Resolver`. La causa è che la maggior parte dei metodi coinvolti richiede come preconditione oggetti complessi (`Schema`, `GenericData`) che Randoop non è in grado di costruire autonomamente tramite reflection. I test generati seguono quasi tutti lo stesso pattern: costruzione di uno `Schema` vuoto o con argomenti `null`, invocazione di `resolve()` e cattura dell'eccezione risultante (tipicamente `NullPointerException`), raggiungendo il metodo solo al punto di ingresso senza mai esercitarne la logica decisionale interna.

Dal punto di vista qualitativo, i test generati utilizzano nomi sequenziali privi di significato semantico (ad esempio `test1`, `test2`), senza commenti né documentazione, con una struttura lineare di chiamate priva di separazione logica: comprendere l'intento di un singolo caso di test richiede la lettura dell'intera implementazione.

Generazione tramite LLM Sono stati utilizzati diversi prompt per sperimentare la generazione di test da parte di un LLM. L'IA utilizzata è stata Gemini, somministrando 3 prompt con quantità di contenuto crescente. Il testo completo dei tre prompt è riportato nella **tabella 9**; la **tabella 10** ne riassume i risultati numerici, mentre la **fig. 4** mostra il dettaglio di copertura per metodo della suite risultante (Prompt 2 e 3 combinati).

Il primo prompt, formulato senza fornire il codice sorgente della classe, ha prodotto codice non compilabile: il modello ha introdotto due classi di eccezione inesistenti nel progetto e un nome di classe errato, presumibilmente per analogia con librerie di validazione più comuni. Il secondo prompt, che includeva il codice sorgente completo, non ha prodotto alcuna allucinazione sui nomi delle API utilizzate, richiedendo solo l'appiattimento di una struttura basata su `@Nested` (incompatibile con la configurazione Surefire del progetto): dopo questa correzione, tutti i 45 test generati sono risultati passanti. Il terzo prompt, che aggiungeva vincoli di progetto espliciti (assenza di `@Nested`, stile di formattazione, categorie di risoluzione da coprire sistematicamente), ha prodotto 13 test integrabili senza alcuna correzione.

Il salto di qualità più rilevante si osserva tra il primo e il secondo prompt: la disponibilità del codice sorgente elimina quasi completamente le allucinazioni sui nomi delle API. Il passaggio dal secondo al terzo prompt migliora invece principalmente l'*integrabilità* del codice generato, riducendo a zero gli interventi manuali necessari, più che la correttezza di base del codice prodotto.

EvoSuite EvoSuite è uno strumento per la generazione automatica di test guidata dalla copertura, eseguito con JDK 11 per lo stesso motivo riportato in precedenza per Randoop. La generazione è stata eseguita con un budget di 60 secondi, producendo 78 test. La **tabella 11** riassume i risultati, e la **fig. 5** ne mostra il dettaglio per metodo.

Durante la generazione, EvoSuite ha segnalato un test instabile (*flaky*): un valore non deterministico utilizzato internamente nel nome di uno schema non risolto varia tra un'esecuzione e l'altra.

La copertura, misurata isolatamente con JaCoCo, risulta pari all'88.5% statement e all'84.2% branch — la più alta tra le tre tecniche automatiche applicate a `Resolver`, risultato atteso poiché EvoSuite è l'unica delle tre esplicitamente guidata dal criterio di copertura durante la generazione.

4.2 Test automatici su Utf8

Randoop Randoop è stato eseguito su `Utf8` con la stessa configurazione già descritta per `Resolver` (versione 4.3.3, JDK 11, budget di 60 secondi), generando 2841 test, nessuno dei quali error-revealing. La **tabella 8** riporta i risultati numerici completi per entrambe le classi; la **fig. 6** mostra il dettaglio per metodo.

A differenza di `Resolver`, su `Utf8` Randoop ottiene una copertura molto più alta (79% statement, 75% branch). Il motivo è speculare a quanto osservato in precedenza: `Utf8` accetta input primitivi semplici (`String`, `byte[]`, interi), che Randoop riesce a costruire facilmente tramite reflection, senza dover generare oggetti compositi complessi come `Schema` e `GenericData`. Questo contrasto conferma empiricamente che il limite riscontrato su `Resolver` non è generale allo strumento, ma specifico ai domini con oggetti compositi articolati.

Generazione tramite LLM Sono stati sperimentati gli stessi tre tipi di interrogazioni già utilizzati per `Resolver`, con Gemini come modello. Il testo completo dei tre prompt è riportato nella **tabella 12**; la **tabella 13** ne riassume i risultati numerici, mentre la **fig. 7** mostra il dettaglio di copertura per metodo.

A differenza di `Resolver`, il primo prompt (senza codice sorgente) non ha prodotto allucinazioni evidenti sui nomi dei metodi: un'ipotesi plausibile è che `Utf8`, essendo una classe pubblica ampiamente documentata e utilizzata negli esempi Avro, sia più "nota" al modello dal materiale di addestramento rispetto a una classe più interna come `Resolver`. Il secondo prompt ha prodotto 23 test, tutti passanti dopo l'appiattimento

della struttura `@Nested`. Il terzo prompt ha prodotto 17 test, di cui 16 integrabili senza correzioni: un singolo test è stato rimosso perché utilizzava un costruttore `package-private` non accessibile dal package in cui il test era stato collocato.

La copertura della suite risultante (Prompt 2 e 3 combinati), misurata con JaCoCo, risulta pari al 97.1% statement e al 95.5% branch — la più alta tra le tre tecniche automatiche applicate a `Utf8`.

EvoSuite EvoSuite è stato eseguito su `Utf8` con la stessa configurazione già descritta per `Resolver` (JDK 11, budget di 60 secondi), producendo 63 test. La **tabella 14** riassume i risultati, e la **fig. 8** ne mostra il dettaglio per metodo.

La copertura, misurata isolatamente con JaCoCo, risulta pari al 95.2% statement e al 95.5% branch, sensibilmente più alta di quella ottenuta su `Resolver` (88.5%/84.2%) con lo stesso budget di ricerca. La differenza è coerente con quanto già osservato per le altre tecniche automatiche: `Utf8`, essendo una classe più piccola e con una logica meno ramificata rispetto a `Resolver`, risulta più facilmente esplorabile a parità di tempo di generazione.

4.3 Integrazione tecnica di EvoSuite

L'integrazione di EvoSuite nel ciclo di build ha richiesto la risoluzione di diversi problemi tecnici, documentati di seguito per completezza metodologica.

Compatibilità del bytecode in fase di generazione La libreria di analisi bytecode inclusa in EvoSuite 1.2.0 (ASM) non è in grado di interpretare correttamente classi compilate per Java 21 (il JDK di default dell'ambiente di sviluppo), né alcune dipendenze del progetto distribuite come *multi-release JAR* (contenenti più versioni di bytecode nello stesso archivio). La generazione è stata quindi eseguita compilando il progetto con `-release 11` e utilizzando esplicitamente il binario Java 11 per lanciare EvoSuite, dopo aver rimosso dal classpath le voci `META-INF/versions/*` dei JAR multi-release interessati.

Dipendenza runtime non disponibile L'esecuzione (non la sola generazione) dei test prodotti da EvoSuite richiede la libreria `evosuite-standalone-runtime`, non pubblicata su Maven Central nella versione necessaria. È stato aggiunto uno step dedicato nella pipeline di CI che scarica il JAR dalla release ufficiale su GitHub e lo installa nel repository Maven locale della macchina di build (`mvn install:install-file`) prima della compilazione, garantendo la riproducibilità della build anche in un ambiente pulito.

Restrizioni di sicurezza della JVM A partire da Java 18, la chiamata a `System.setSecurityManager(...)` — utilizzata dal meccanismo di sandboxing di EvoSuite — è vietata senza un'esplicita autorizzazione. È stato necessario aggiungere il flag `-Djava.security.manager=allow` all'`argLine` di Surefire; poiché tale configurazione non viene ereditata tra il `pom.xml` padre e quello del modulo (gli elementi di configurazione a valore singolo si sovrascrivono, non si concatenano), il flag deve essere dichiarato nel `pom.xml` del modulo effettivamente eseguito.

Verifica del bytecode a runtime L'esecuzione dei test EvoSuite su JDK 21 produceva un errore di verifica del bytecode (`VerifyError`), assente su JDK 11: il bytecode strumentato da EvoSuite risulta incompatibile con le regole di verifica più recenti introdotte nelle JVM successive alla 11. È stato configurato un elemento `<jdkToolchain>` nel plugin Surefire, per forzare l'esecuzione dei test su JDK 11 indipendentemente dal JDK usato per compilare il resto del progetto.

Inquinamento di stato tra classi di test L'aggiunta dei test EvoSuite ha esposto, in modo intermittente e dipendente dall'ordine di esecuzione, un conflitto a livello di `ServiceLoader`: l'strumentazione di EvoSuite carica alcuni provider SPI da un classloader diverso da quello atteso dall'interfaccia, corrompendo lo stato statico condiviso (in particolare la classe `LogicalTypes`) per l'intera durata del processo Surefire. La causa è stata individuata in risorse `META-INF/services` superflue, relative a test nativi del progetto già esclusi dall'esecuzione (Sezione 2.3): la loro rimozione dai `testResources` ha risolto il problema alla radice.

5 Validazione della qualità dei test

Le sezioni precedenti hanno applicato quattro famiglie di test alle due classi selezionate, senza tuttavia confrontarne sistematicamente l'efficacia. Questo capitolo affronta tale confronto su tre livelli complementari: quanto codice viene effettivamente eseguito da ciascuna famiglia di test (metriche di adeguatezza); quanto i test sono in grado di rilevare difetti introdotti artificialmente nel codice (mutation testing); e quale stima di affidabilità si può derivare dall'insieme finale di test disponibile (reliability).

5.1 Metriche di adeguatezza

Per confrontare quantitativamente l'efficacia delle quattro famiglie di test applicate a ciascuna classe (test manuali, Randoop, LLM, EvoSuite), sono state scelte due metriche di adeguatezza control-flow: *Statement Coverage* e *Branch Coverage*. La scelta di queste due metriche consente un confronto omogeneo con i dati già raccolti per EvoSuite nel capitolo precedente. Per ciascuna combinazione famiglia/classe, la copertura è stata misurata isolando l'esecuzione della sola famiglia in questione con JaCoCo, azzerando i dati di copertura precedenti prima di ogni misurazione. La **tabella 15** riporta i risultati completi.

EvoSuite risulta la famiglia più efficace su entrambe le classi (88.5% e 95.2% statement coverage rispettivamente), risultato atteso poiché è l'unica delle quattro tecniche esplicitamente guidata dal criterio di copertura durante la generazione. La suite manuale (BB) è invece la meno efficace in termini di copertura strutturale (42.6% e 66.7%): questo non indica una debolezza del category-partition come tecnica, ma riflette il fatto che le categorie individuate mirano a rappresentare gli scenari d'uso funzionali della classe, non a massimizzare la copertura di ogni riga o diramazione del codice.

Il dato più rilevante emerso dal confronto è il comportamento di Randoop, che presenta un divario estremo tra le due classi: 79.0% statement coverage su `Utf8`, contro appena l'1.6% su `Resolver`. La causa è che Randoop costruisce gli input tramite reflection, riuscendo a generare valori primitivi (`String`, `byte[]`) sufficienti a esercitare efficacemente `Utf8`, ma non riuscendo a costruire per via casuale oggetti `Schema` e `GenericData` semanticamente validi, necessari per attraversare la logica di `Resolver.resolve()`. Questo risultato è coerente con un limite noto del random testing basato su reflection, quando applicato a domini con oggetti composti articolati.

Gli LLM si posizionano stabilmente al secondo posto su entrambe le classi (75.4%/68.4% su `Resolver`, 97.1%/95.5% su `Utf8`), superando sia la suite manuale sia Randoop: avendo accesso al codice sorgente e a vincoli di progetto espliciti, la generazione tramite LLM produce test sistematici che si avvicinano alla qualità di un approccio guidato dalla copertura, pur senza esserlo esplicitamente.

5.2 Mutation testing con PIT

Il mutation testing consiste nell'introdurre piccole modifiche artificiali nel bytecode compilato (*mutanti*) e verificare se i test esistenti le rilevano: un mutante è detto *ucciso* se almeno un test fallisce in sua presenza, *sopravvissuto* altrimenti. A differenza delle metriche di adeguatezza discusse in precedenza, questa tecnica misura non solo quanto codice viene eseguito dai test, ma quanto efficacemente le loro asserzioni sono in grado di distinguere un comportamento corretto da uno alterato. L'analisi è stata condotta con PIT sulla sola suite di test manuali (BB), come richiesto dal punto 4b della consegna.

Problemi di integrazione risolti Sono emersi due problemi tecnici nell'integrazione di PIT nel progetto. Il primo (errore `MINION_DIED`) era dovuto al placeholder `${argLine}` di Surefire, valorizzato solo quando il goal `jacoco:prepare-agent` viene eseguito nella stessa invocazione Maven: lanciando PIT da solo, il placeholder restava non risolto, causando un errore nei processi di misurazione di PIT. È stata dichiarata una property di progetto `<argLine></argLine>` vuota per risolvere il problema. Il secondo problema riguardava una mutation coverage 0%/0% falsa su `Utf8`: PIT esegue di base solo test JUnit 3/4, mentre `TestUtf8CategoryPartition` è scritta in JUnit 5. È stato necessario aggiungere la dipendenza `pitest-junit5-plugin` al plugin.

Risultati iniziali La **tabella 16** riassume i risultati numerici, mentre la **fig. 9** mostra il report generato da PIT. Sulla suite manuale di `Resolver`, PIT non ha rilevato alcun mutante sopravvissuto tra quelli coperti (Test Strength 100%). Su `Utf8` sono invece emersi 12 mutanti sopravvissuti, concentrati sui metodi `equals()`, `hashCode()`, `toString()` e `setByteLength()`.

Test di rinforzo (MT) Sono stati scritti 14 nuovi test manuali mirati a uccidere i 12 mutanti sopravvissuti su `Utf8`. Durante la loro progettazione sono emerse due scoperte metodologiche: un primo tentativo di verifica tramite confronto per identità di riferimento (`assertSame`) su una stringa vuota non uccideva il mutante corrispondente, poiché la JDK ottimizza tale caso restituendo comunque il letterale internato indipendentemente dal ramo di codice eseguito – risolto verificando lo stato interno tramite reflection; inoltre, un test già presente nella suite BB non esercitava effettivamente il metodo `equals()` di `Utf8`, poiché il contratto di JUnit 5 poteva invocare `String.equals()` anziché `Utf8.equals()` per l'argomento fornito.

Dei 12 mutanti target, 7 sono stati uccisi dai nuovi test. I restanti 5 sono stati argomentati come mutanti *equivalenti*: i due percorsi di calcolo di `equals()` e `hashCode()` (un ramo ottimizzato tramite `Arrays.equals/Arrays.hashCode` per array esattamente dimensionati, e un loop manuale byte-per-byte per gli altri casi) calcolano matematicamente lo stesso risultato per qualunque input valido, verificato empiricamente con test mirati proprio al confine tra i due rami. Dopo l'integrazione dei test MT, la mutation coverage di `Utf8` è salita al 56% (33/59), con Test Strength dell'87% (33/38).

5.3 Stima della reliability

La reliability di un sistema può essere stimata come $reliability = 1 - PFD$, dove PFD (*probability of failure on demand*) rappresenta la probabilità che un'operazione fallisca, assumendo un *profilo operativo* che assegna una probabilità di utilizzo a ciascuna operazione del sistema. In questo lavoro è stato adottato un profilo operativo a probabilità uniforme.

La scelta di quali operazioni considerare è stata guidata da un criterio di rappresentatività: anziché trattare ogni singolo test generato automaticamente come un'operazione a sé – scelta che sovra-rappresenterebbe input casuali o degeneri rispetto a scenari d'uso plausibili, specialmente per i test prodotti da Randoop – sono state utilizzate le categorie della suite di test manuali (BB) come operazioni rappresentative, ciascuna pesata uniformemente $1/N$.

Su `Resolver`, le 8 categorie individuate risultano tutte associate a test passanti, con una Test Strength del 100% rilevata dal mutation testing: la reliability stimata è pertanto pari a 1.0. Analogamente, su `Utf8` le 6 categorie risultano tutte associate a test passanti (suite BB e MT complessivamente), con una reliability stimata anch'essa pari a 1.0.

Questo risultato va qualificato: rappresenta la reliability osservata rispetto al profilo operativo definito dalle categorie testate, non una garanzia di correttezza sull'intero dominio di input della classe. L'analisi di mutation testing (Sezione 5.2) ha già mostrato che una parte dei mutanti generati da PIT non è coperta da alcun test della suite manuale: per le porzioni di codice corrispondenti non esiste evidenza sperimentale, e la stima di reliability vale quindi solo entro i limiti del profilo operativo adottato.

6 Refactoring e validazione incrociata

Il punto 5 della consegna richiede di generare, tramite un modello linguistico, quattro varianti rifattorizzate (C1-C4) di ciascuna classe, con contesto crescente: C1 senza contesto, C2 con la suite manuale (BB), C3 con l'aggiunta dei gap di coverage, C4 con l'aggiunta dei test di mutation reinforcement (MT), dove disponibili. Per ciascuna variante tutte le famiglie di test vengono rilanciate, misurando pass/fail, copertura e mutation score rispetto a C0. L'obiettivo è verificare se un refactoring assistito da LLM introduce regressioni, e quali tecniche di test siano effettivamente in grado di rilevarle: un refactoring che compili e superi tutti i test esistenti non è automaticamente un refactoring sicuro, se nessuna di quelle suite esercita la porzione di codice realmente modificata. Per questo motivo, oltre all'esito aggregato pass/fail, viene riportato per ciascuna classe anche il comportamento delle singole famiglie di test: è a questo livello di dettaglio, e non nel solo dato complessivo, che emergono le differenze più interessanti tra le tecniche.

6.1 Refactoring di `Resolver`

Sono state generate quattro varianti rifattorizzate di `Resolver`. C1 (nessun contesto) rinomina `ErrorAction.error` in `errorType`, sostituisce `Promote.isValid` con una tabella statica e riscrive `toString()`, `noReorder()` e `firstMatchingBranch` con `stream` e tabelle statiche. C2 (+BB) applica le stesse modifiche di C1 ad eccezione del rename del campo `error`, che il modello evita non appena riceve in input la suite manuale. C3 (+gap coverage) si limita al solo refactoring di `Promote.isValid`, l'unica modifica in una zona ben coperta dalla suite manuale. C4 (+MT) è identica a C3, poiché `Resolver` non ha una suite MT dedicata (Test Strength già 100% con la sola suite BB).

La **tabella 17** riassume l'esito per famiglia:

- **BB (32 test)**: FAIL compilazione su C1, per il riferimento diretto al campo `error` in un test di verifica su `ErrorAction`. 32/32 da C2 in poi, senza modifiche al codice di test.
- **Randoop (campione di 500 test)**: FAIL compilazione su C1, stesso motivo. Su C2, 5 test su 500 falliscono l'esecuzione: è qui che è stato scoperto un bug di ricorsione, introdotto semplificando la chiamata `Resolver.resolve(...)` in `resolve(...)` – la chiamata non qualificata veniva risolta al metodo statico `resolve` della sottoclasse (`RecordAdjust`, `WriterUnion` o `ReaderUnion` dichiarano ciascuna la propria versione con la stessa firma) anziché al dispatcher esterno. Corretto ripristinando la qualificazione esplicita; 500/500 su C2-C4.
- **LLM (58 test)**: FAIL compilazione su C1; 58/58 da C2 in poi.
- **EvoSuite (78 test)**: PASS su tutte le varianti, incluso C1 – genera le proprie sequenze per riflessione sui soli metodi pubblici e non referencia mai un campo interno per nome, quindi il rename non lo tocca.

Nessuna delle quattro famiglie rileva il refactoring di `toString()`, `noReorder()` e `firstMatchingBranch` su C2: nessuna suite costruisce uno scenario con più campi obbligatori mancanti insieme, l'unico caso in cui la modifica sarebbe osservabile dall'esterno. È un punto cieco condiviso da tutte le tecniche, non solo dalla suite manuale.

La **tabella 18** riporta i delta rispetto a C0. Copertura: 84.7%/68.1% su C1 (solo suite ES compilante, quindi non comparabile 1:1 con le altre varianti), 93.1%/83.8% su C2, 92.2%/82.8% su C3-C4. Mutation score: N/A su C1 (nessuna suite eleggibile per PIT resta compilante), 51% invariato da C2 in poi. Com-

plexità (proxy LOC/punti di decisione, in assenza di un'analisi SonarCloud diretta sulle varianti): -15% su C1-C2, dove il refactoring più aggressivo introduce tabelle statiche e stream al posto di cicli e confronti a cascata; -6% su C3-C4, più conservative.

Va inoltre segnalato un limite tecnico già discusso in Sezione 5.2: PIT ed EvoSuite sono strutturalmente incompatibili (conflitto tra il meccanismo di *HotSwap* di PIT e l'instrumentazione a runtime di EvoSuite), per cui il mutation score qui riportato esclude sistematicamente la suite EvoSuite.

6.2 Refactoring di Utf8

Il design dei refactor per Utf8 si è basato su un rischio reale individuato nella classe durante l'analisi del codice sorgente: `equals()` e `hashCode()` implementano un percorso ottimizzato per array esattamente dimensionati, che richiede una guardia esplicita sulla lunghezza effettiva del buffer; inoltre, i metodi `set(String)` e `set(Utf8)` sono esercitati dalle famiglie LLM, Randoop ed EvoSuite, ma da nessun test delle famiglie BB e MT – un possibile punto debole in caso di refactoring proprio su quei metodi.

C1 (nessun contesto) rinomina `set` in `setFromString/setFromUtf8`, unifica `equals()` in un'unica chiamata a `Arrays.equals` e rimuove per errore la guardia `bytes.length == length` da `hashCode()`, introducendo un bug latente sui buffer con capacità residua. C2 (+BB) annulla il rename di `set`, ma il bug di `hashCode()` resta: la suite BB non esercita mai lo scenario di buffer con capacità residua. C3 (+gap coverage, che segnala esplicitamente `setByteLength`, `equals` e `hashCode` come aree a rischio) diventa conservativa, ripristina `equals()` e `hashCode()` identici all'originale e sposta il refactoring su `compareSequences()`. C4 (+MT) reintroduce con fiducia il merge di `equals()` – i test di boundary della suite MT sui confini di lunghezza 7/8 lo confermano equivalente – e rende esplicita, anziché rimuovere, la guardia di `hashCode()` tramite un metodo estratto `isExactlySized(...)`.

La **tabella 19** riassume l'esito per famiglia:

- **BB (35 test)**: 35/35 su tutte le varianti. La suite non referencia mai `set` se non tramite l'interfaccia pubblica, e il rename viene comunque annullato dal modello già in C2.
- **MT (14 test)**: 13/14 su C1-C2 – il fallimento è esattamente il test che verifica il bug di `hashCode()`; 14/14 su C3-C4, dove la guardia viene ripristinata o resa esplicita.
- **Randoop (campione di 500 test)**: FAIL su C1 (rename di `set`); 500/500 da C2. Il campionamento di input primitivi non costruisce mai, nemmeno per caso, un'istanza tramite il costruttore `package-private Utf8(String, int)` necessario a esporre il bug di `equals()`.
- **LLM (40 test)**: FAIL su C1; 40/40 da C2, stesso limite di Randoop – costruisce le proprie istanze solo tramite costruttori pubblici.
- **EvoSuite (63 test)**: FAIL su C1; 61/63 su C2 – le 2 failure sono la prima evidenza del bug di `equals()`, raggiunto costruendo per reflection un'istanza con lunghezza negativa tramite `Utf8(String, int)`, accessibile a EvoSuite perché il test generato si trova nello stesso package. 63/63 su C3 (refactoring conservativo); 62/63 su C4, poiché il merge di `equals()` viene reintrodotta.

I due difetti reali introdotti dal refactoring sono quindi complementari e di difficoltà molto diversa: il bug di `hashCode()` richiede solo API pubbliche, alla portata di qualunque tecnica scelga lo scenario giusto (in pratica solo MT lo fa); il bug di `equals()` richiede l'accesso a un costruttore a visibilità ristretta, alla portata solo di una tecnica basata su reflection strutturale come EvoSuite. Nessuna delle due tecniche sarebbe stata sufficiente da sola.

La **tabella 20** riporta i delta rispetto a C0. Copertura: 68.7%/47.2% su C1 (solo BB+MT compilanti), 100%/97.2% su C2, 100%/97.7% su C3, 100%/97.5% su C4. Mutation score: N/A su C1-C2, poiché la suite MT non è interamente verde – preconditione richiesta da PIT; 86% su C3, invariato rispetto a C0; 85% su C4, lieve calo compatibile con il punto di decisione aggiuntivo introdotto da `isExactlySized(...)`. Complessità: -11% punti di decisione su C1, la riduzione più marcata di tutte le varianti, ma anche l'unica con un bug reale non rilevato dalla maggioranza delle famiglie – una controprova diretta del compromesso tra aggressività del refactoring e sicurezza comportamentale.

Leggibilità dei test Un'osservazione qualitativa, verificata direttamente sul codice sorgente generato: i test BB, MT e LLM hanno nomi di metodo descrittivi e un commento esplicito di intento per ogni caso (es. `testSetByteLengthExpansion`, con `@DisplayName` per la suite LLM); Randoop ed EvoSuite usano invece naming puramente sequenziale (`test0001`, `test00`) e nessun commento sul comportamento verificato – gli unici commenti presenti in Randoop sono generati automaticamente dallo strumento per segnalare un'eccezione attesa, non per documentare l'intento. Questa differenza, verificata restando costante su tutte le varianti (la stessa suite fisica viene ripuntata sul codice variante di turno, anziché rigenerata da zero come nel report di riferimento della stessa consegna), ha una ricaduta pratica diretta: i due interventi di correzione descritti in Sezione 5.2 sono stati diagnosticabili solo perché il test coinvolto era leggibile; sarebbe stato impraticabile farlo su una suite con la leggibilità di Randoop o EvoSuite, dove l'intento originale del test non è

mai esplicitato.

6.3 Sintesi

I risultati ottenuti sulle due classi convergono verso la stessa conclusione, pur mostrando due facce diverse dello stesso problema. Su `Resolver`, nessuna delle quattro tecniche rileva il refactoring di `toString()`: è uno stesso punto cieco condiviso da tutte, dovuto all'assenza di scenari con più campi obbligatori mancanti insieme. Su `Utf8`, invece, il refactoring introduce due difetti reali e distinti, e ciascuno risulta visibile a una sola famiglia di test su cinque: il mutation testing individua un valore di calcolo errato raggiungibile con le sole API pubbliche, mentre EvoSuite individua una differenza di comportamento raggiungibile solo sfruttando uno stato interno costruibile esclusivamente tramite reflection o un costruttore riservato.

Questi risultati confermano empiricamente che nessuna singola tecnica di test è sufficiente da sola a proteggere da regressioni introdotte da un refactoring assistito da un modello linguistico con contesto parziale. Ciascuna tecnica esplora lo spazio degli input secondo un principio proprio – rappresentatività funzionale per i test manuali, sensibilità a mutazioni specifiche per il mutation testing, casualità guidata dal feedback per Randoop, plausibilità sintattica per gli LLM, massimizzazione della copertura strutturale per EvoSuite – e nessuno di questi principi, preso isolatamente, garantisce la copertura degli scenari esplorati dagli altri. La combinazione di più tecniche complementari, unita a un'analisi mirata di copertura e mutation score per guidare eventuali interventi correttivi, risulta pertanto necessaria per validare con affidabilità un refactoring del codice.

Questo capitolo chiude idealmente il percorso avviato al Capitolo 3: la suite manuale, la più debole in termini di copertura strutturale grezza, si è rivelata qui l'unica in grado di intercettare un difetto (`hashCode()`) proprio grazie ai casi limite individuati con la tecnica Category-Partition, mentre le tecniche automatiche più esplorative (EvoSuite) hanno raggiunto scenari – come l'accesso a un costruttore riservato – che nessun progettista di test, umano o guidato da un prompt, avrebbe verosimilmente pensato di includere. Il valore aggiunto di un processo di verifica software non risiede quindi in una singola tecnica "migliore delle altre", ma nella loro combinazione mirata e nella capacità di interpretare criticamente i dati di copertura e mutazione che ciascuna produce.

7 Conclusioni

Il progetto ha applicato un insieme di tecniche di verifica software, manuali e automatiche, a due classi di un fork di Apache Avro: `Resolver`, che implementa la logica di risoluzione tra schema *writer* e *reader*, e `Utf8`, che rappresenta una sequenza di testo codificata direttamente in byte UTF-8. Per ciascuna classe sono stati sviluppati test manuali basati su Category-Partition, test generati automaticamente tramite random testing (Randoop), generazione assistita da un modello linguistico (LLM) e generazione guidata dalla copertura (EvoSuite); la qualità dei test è stata validata tramite metriche di adeguatezza, mutation testing (PIT) e una stima di reliability; infine, entrambe le classi sono state sottoposte a un refactoring in quattro varianti con contesto crescente, rilanciando tutte le famiglie di test sviluppate per verificarne la tenuta.

I risultati più rilevanti emersi dal lavoro sono tre. In primo luogo, il random testing basato su reflection si è dimostrato fortemente dipendente dalla complessità del dominio di input: efficace su `Utf8` (79% statement coverage), quasi inefficace su `Resolver` (1.6%), a causa dell'incapacità di costruire autonomamente oggetti Schema semanticamente validi. In secondo luogo, la generazione tramite LLM ha mostrato un netto miglioramento di qualità in funzione del contesto fornito: da un tasso di allucinazione elevato in assenza del codice sorgente, fino a un'integrazione pressoché immediata quando il prompt include anche i vincoli specifici del progetto. In terzo luogo, e più significativamente, l'analisi del refactoring ha dimostrato empiricamente che nessuna singola tecnica di test è sufficiente a garantire la correttezza di una modifica al codice: su `Utf8` sono stati introdotti due difetti reali e distinti, ciascuno rilevabile da una sola delle cinque famiglie di test disponibili, a conferma che tecniche complementari esplorano porzioni diverse e non sovrapponibili dello spazio di input.

Il lavoro presenta alcuni limiti. La valutazione degli *smell* di codice nelle varianti rifattorizzate si è basata su un proxy locale (numero di righe di codice e punti di decisione), in assenza di un'analisi diretta con SonarCloud sulle versioni rifattorizzate. La validazione delle varianti rispetto alla famiglia Randoop ha utilizzato un campione della suite generata, anziché la sua totalità, per ragioni di tempo di esecuzione. Infine, è stato riscontrato un limite tecnico strutturale tra PIT ed EvoSuite (conflitto tra il meccanismo di *HotSwap* e l'strumentazione a runtime), che ha impedito di calcolare il mutation score della suite EvoSuite in tutte le configurazioni analizzate.

Sviluppi futuri del lavoro potrebbero includere l'estensione dell'analisi di qualità del codice alle varianti rifattorizzate tramite SonarCloud, l'esecuzione della validazione Randoop sull'intera suite generata anziché su un campione, e un approfondimento della causa tecnica del conflitto tra PIT ed EvoSuite, valutando eventuali configurazioni alternative del motore di mutazione in grado di superare l'incompatibilità riscontrata.

A Tabelle, figure e listati

Sottoclasse Action	Condizione di attivazione	Effetto	Esempio pratico
DoNothing	Writer e reader hanno lo stesso tipo	Nessuna trasformazione	INT → INT
Promote	Tipi numerici compatibili ma diversi	Conversione automatica sicura	INT → LONG
ErrorAction	Incompatibilità irrisolvibile	Segnalazione di errore (ErrorType)	FIXED di 16 byte → FIXED di 32 byte
Container	Tipo ARRAY o MAP	Risoluzione ricorsiva di ogni elemento	array<int> → array<long>
EnumAdjust	Tipo ENUM	Rimappatura dei simboli tra writer e reader	simboli riordinati, stessi nomi
RecordAdjust (+ Skip)	Tipo RECORD	Risoluzione campo per campo; campi rimossi ignorati (Skip), campi aggiunti riempiti con default	campo obsoleto rimosso; nuovo campo con default
ReaderUnion	Reader è UNION, writer no	Ricerca del ramo della union compatibile con il tipo scritto	STRING → ["null", "string"]
WriterUnion	Writer è UNION	Risoluzione di ogni possibile ramo scritto rispetto al reader	["int", "long"] → LONG

Table 1: Le otto categorie di risoluzione gestite da Resolver.Action e relativi esempi.

Categoria (classe di equivalenza)	Valori rappresentativi utilizzati nei test
DoNothing	NULL, BOOLEAN, INT, LONG, FLOAT, DOUBLE, STRING, BYTES (schema writer e reader identici, un test per ciascun tipo primitivo)
Promote	Valide: INT→LONG, INT→FLOAT, INT→DOUBLE, LONG→FLOAT, LONG→DOUBLE, FLOAT→DOUBLE, STRING→BYTES, BYTES→STRING. Boundary (non valide): LONG→INT, DOUBLE→FLOAT, STRING→INT
ErrorAction	FIXED con nomi diversi a parità di dimensione; FIXED con dimensioni diverse a parità di nome; combinazioni di tipi primitivi incompatibili (es. BOOLEAN→STRING)
Container	array<int>→array<long> (con promozione dell'elemento); map<string>→map<string> (elementi identici)
EnumAdjust	Simboli identici, stesso ordine; simboli riordinati a parità di nomi; enum con nome diverso tra writer e reader (boundary verso ErrorAction)
RecordAdjust	Record con campi identici; campo aggiunto nel writer, assente nel reader (Skip); campo assente nel writer, presente nel reader con valore di default; campo assente nel writer, presente nel reader senza default (boundary verso ErrorAction)
ReaderUnion	Schema writer non-union verso union ["null", "string"], con match diretto; con match tramite promozione numerica; nessun ramo compatibile (boundary verso ErrorAction)
WriterUnion	Schema writer union ["int", "long"] verso reader LONG, con più rami risolti singolarmente

Table 2: Valori rappresentativi utilizzati per ciascuna categoria di test su Resolver.

Table 3: Elenco completo dei 32 test manuali su Resolver.

ID	Categoria	Input (writer → reader)	Assert atteso
T01	DoNothing	NULL → NULL	istanza di DoNothing
T02	DoNothing	BOOLEAN → BOOLEAN	istanza di DoNothing
T03	DoNothing	INT → INT	istanza di DoNothing
T04	DoNothing	LONG → LONG	istanza di DoNothing
T05	DoNothing	FLOAT → FLOAT	istanza di DoNothing

ID	Categoria	Input (writer → reader)	Assert atteso
T06	DoNothing	DOUBLE → DOUBLE	istanza di DoNothing
T07	DoNothing	STRING → STRING	istanza di DoNothing
T08	DoNothing	BYTES → BYTES	istanza di DoNothing
T09	Promote	INT → LONG	istanza di Promote
T10	Promote	INT → FLOAT	istanza di Promote
T11	Promote	INT → DOUBLE	istanza di Promote
T12	Promote	LONG → FLOAT	istanza di Promote
T13	Promote	FLOAT → DOUBLE	istanza di Promote
T14	Promote	STRING → BYTES	istanza di Promote
T15	Promote	BYTES → STRING	istanza di Promote
T16	ErrorAction	LONG → INT (promozione all'indietro)	istanza di ErrorAction
T17	ErrorAction	STRING → INT	istanza di ErrorAction
T18	ErrorAction	BOOLEAN → STRING	istanza di ErrorAction
T19	FIXED	FIXED("F",4) → FIXED("F",4) (nome e size uguali)	istanza di DoNothing
T20	FIXED	FIXED("F1",4) → FIXED("F2",4) (nomi diversi)	ErrorAction con NAMES_DONT_MATCH
T21	FIXED	FIXED("F",4) → FIXED("F",8) (size diverse)	ErrorAction con SIZES_DONT_MATCH
T22	Container	array<int> → array<int>	istanza di Container
T23	Container	map<string> → map<string>	istanza di Container
T24	Container	array<int> → array<long>	Container con elementAction di tipo Promote
T25	EnumAdjust	enum E con simboli A,B,C identici	istanza di EnumAdjust
T26	EnumAdjust	enum E1 → enum E2 (nomi diversi)	istanza di ErrorAction
T27	RecordAdjust	record R con campo a identico	istanza di RecordAdjust
T28	RecordAdjust	writer con campi a,b, reader con solo a	RecordAdjust contenente uno Skip per il campo b
T29	RecordAdjust	writer con solo a, reader con a,b senza default	istanza di ErrorAction
T30	ReaderUnion	INT → union [NULL,INT]	istanza di ReaderUnion
T31	WriterUnion	union [NULL,INT] → INT	istanza di WriterUnion
T32	Union (errore)	STRING → union [NULL,INT] (nessun ramo compatibile)	istanza di ErrorAction

Table 5: Elenco completo dei 35 test manuali su Utf8.

ID	Categoria	Input / Azione	Assert atteso
T1	Costruzione	new Utf8()	getByteLength()==0
T2	Costruzione	new Utf8("")	getByteLength()==0
T3	Costruzione	new Utf8("hello")	toString()=="hello"; getByteLength()==5
T4	Costruzione	new Utf8("caffè")	toString() uguale all'originale; getByteLength() pari ai byte UTF-8
T5	Costruzione	new Utf8((String) null)	lancia NullPointerException
T6	Costruzione	new Utf8(original) (copia)	uguale a original; istanza distinta
T7	Costruzione	new Utf8(new byte[0])	getByteLength()==0
T8	Costruzione	new Utf8("abc".getBytes(US_ASCII))	toString()=="abc"
T9	Uguaglianza	u.equals(u)	true
T10	Uguaglianza	due istanze con contenuto "hello"	equals=true; hashCode uguali
T11	Uguaglianza	"abc" vs "abd"	equals=false
T12	Uguaglianza	"abc" vs "abcdef"	equals=false
T13	Uguaglianza	confronto con null	equals=false (nessuna eccezione)
T14	Uguaglianza	confronto con una String	equals=false
T15	Ordinamento	u.compareTo(u)	0
T16	Ordinamento	stesso contenuto	0

ID	Categoria	Input / Azione	Assert atteso
T17	Ordinamento	"abc" vs "abd"	segno negativo/positivo coerente nei due versi
T18	Ordinamento	"ab" vs "abc" (prefisso)	il più corto precede
T19	Gestione lunghezza	setByteLength in aumento (2→5)	lunghezza aggiornata a 5; buffer azzerato, non preservato
T20	Gestione lunghezza	setByteLength in diminuzione (5→2)	lunghezza 2; toString()=="he"
T21	Gestione lunghezza	setByteLength invariata (3→3)	toString()=="abc"
T22	Gestione lunghezza	setByteLength(0)	equivalente a new Utf8("")
T23	CharSequence	charAt(0) su "hello"	'h'
T24	CharSequence	charAt(length-1) su "hello"	'o'
T25	CharSequence	charAt(10) su "hi"	lancia IndexOutOfBoundsException
T26	CharSequence	subSequence(2,2) su "hello"	stringa vuota
T27	CharSequence	subSequence(1,4) su "hello"	"ell"
T28	CharSequence	subSequence(4,1) su "hello"	lancia IndexOutOfBoundsException
T29	Serializzazione	round-trip di istanza vuota	uguale a new Utf8("")
T30	Serializzazione	round-trip di "hello"	uguale all'originale
T31	Serializzazione	round-trip di "caffè"	uguale all'originale
T32	getBytesFor	getBytesFor("")	array di lunghezza 0
T33	getBytesFor	getBytesFor("abc")	byte UTF-8 attesi
T34	getBytesFor	getBytesFor("caffè")	byte UTF-8 attesi; lunghezza byte > lunghezza caratteri
T35	getBytesFor	getBytesFor(null)	lancia NullPointerException

Table 6: Elenco completo dei 32 test manuali su Resolver.

ID	Categoria	Input (writer → reader)	Assert atteso
T01	DoNothing	NULL → NULL	istanza di DoNothing
T02	DoNothing	BOOLEAN → BOOLEAN	istanza di DoNothing
T03	DoNothing	INT → INT	istanza di DoNothing
T04	DoNothing	LONG → LONG	istanza di DoNothing
T05	DoNothing	FLOAT → FLOAT	istanza di DoNothing
T06	DoNothing	DOUBLE → DOUBLE	istanza di DoNothing
T07	DoNothing	STRING → STRING	istanza di DoNothing
T08	DoNothing	BYTES → BYTES	istanza di DoNothing
T09	Promote	INT → LONG	istanza di Promote
T10	Promote	INT → FLOAT	istanza di Promote
T11	Promote	INT → DOUBLE	istanza di Promote
T12	Promote	LONG → FLOAT	istanza di Promote
T13	Promote	FLOAT → DOUBLE	istanza di Promote
T14	Promote	STRING → BYTES	istanza di Promote
T15	Promote	BYTES → STRING	istanza di Promote
T16	ErrorAction	LONG → INT (promozione all'indietro)	istanza di ErrorAction
T17	ErrorAction	STRING → INT	istanza di ErrorAction
T18	ErrorAction	BOOLEAN → STRING	istanza di ErrorAction
T19	FIXED	FIXED("F",4) → FIXED("F",4) (nome e size uguali)	istanza di DoNothing
T20	FIXED	FIXED("F1",4) → FIXED("F2",4) (nomi diversi)	ErrorAction con NAMES_DONT_MATCH
T21	FIXED	FIXED("F",4) → FIXED("F",8) (size diverse)	ErrorAction con SIZES_DONT_MATCH
T22	Container	array<int> → array<int>	istanza di Container
T23	Container	map<string> → map<string>	istanza di Container

ID	Categoria	Input (writer → reader)	Assert atteso
T24	Container	array<int> → array<long>	Container con elementAction di tipo Promote
T25	EnumAdjust	enum E con simboli A,B,C identici	istanza di EnumAdjust
T26	EnumAdjust	enum E1 → enum E2 (nomi diversi)	istanza di ErrorAction
T27	RecordAdjust	record R con campo a identico	istanza di RecordAdjust
T28	RecordAdjust	writer con campi a,b, reader con solo a	RecordAdjust contenente uno Skip per il campo b
T29	RecordAdjust	writer con solo a, reader con a,b senza default	istanza di ErrorAction
T30	ReaderUnion	INT → union [NULL,INT]	istanza di ReaderUnion
T31	WriterUnion	union [NULL,INT] → INT	istanza di WriterUnion
T32	Union (errore)	STRING → union [NULL,INT] (nessun ramo compatibile)	istanza di ErrorAction

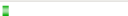
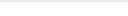
Metodo	Descrizione
<code>Utf8()</code>	Costruttore vuoto.
<code>Utf8(String)</code>	Costruisce l'istanza convertendo la stringa in byte UTF-8.
<code>Utf8(byte[])</code>	Costruisce l'istanza direttamente da un array di byte già in UTF-8.
<code>Utf8(Utf8)</code>	Costruttore di copia.
<code>set(String) / set(Utf8)</code>	Sostituisce il contenuto di un'istanza già esistente, riutilizzando l'array di byte interno quando possibile.
<code>getBytes()</code>	Restituisce l'array di byte interno.
<code>getByteLength()</code> / <code>setByteLength(int)</code>	Leggono e modificano la lunghezza logica di byte effettivamente utilizzati.
<code>equals(Object)</code>	Confronto di uguaglianza, con un percorso ottimizzato (<code>Arrays.equals</code>) per array esattamente dimensionati e un loop manuale byte-per-byte negli altri casi.
<code>hashCode()</code>	Calcolo dell'hash, con la stessa distinzione a due percorsi di <code>equals()</code> .
<code>compareTo(Utf8)</code>	Confronto lessicografico tra due istanze.
<code>toString()</code>	Conversione in <code>String</code> Java (UTF-16), con memorizzazione in cache del risultato.
<code>length()</code> , <code>charAt(int)</code> , <code>subSequence(int, int)</code>	Metodi dell'interfaccia <code>CharSequence</code> .
<code>compareSequences(...)</code> (statico)	Utilità per confrontare <code>CharSequence</code> di tipo eterogeneo (es. <code>Utf8</code> e <code>String</code>).
<code>getBytesFor(String)</code> (statico)	Utilità per estrarre i byte UTF-8 di una stringa.

Table 4: Metodi pubblici principali della classe `Utf8`.

Classe	Numero di test	Statement Coverage	Branch Coverage
<code>Resolver</code>	32	42.6% (26/61)	31.6% (18/57)
<code>Utf8</code>	35	66.7% (70/105)	43.2% (19/44)

Table 7: Copertura strutturale (JaCoCo) della sola suite manuale.

Resolver

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
unionEquiv(Schema, Schema, Map)		4%		2%	22	23	33	36	0	1
resolve(Schema, Schema, GenericData, Map)		94%		89%	2	13	1	22	0	1
Resolver()		0%		n/a	1	1	1	1	1	1
resolve(Schema, Schema, GenericData)		100%		n/a	0	1	0	1	0	1
resolve(Schema, Schema)		100%		n/a	0	1	0	1	0	1
Total	194 of 342	43%	39 of 57	31%	25	39	35	61	1	5

Created with JaCoCo 0.8.12.202403310830

Figure 1: Report JaCoCo della suite manuale su `Resolver`.

Classe	Test generati	Error-revealing	Statement Cov.	Branch Cov.
<code>Resolver</code>	3508	0	1.6%	0%
<code>Utf8</code>	2841	0	79%	75%

Table 8: Risultati della generazione con Randoop (budget 60s), coverage isolata per classe.

Jtf8

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
● compareSequences(CharSequence, CharSequence)		0%		0%	9	9	12	12	1	1
● set(Utf8)		0%		0%	2	2	7	7	1	1
● set(String)		0%		n/a	1	1	8	8	1	1
● equals(Object)		76%		83%	2	7	2	14	0	1
● hashCode()		81%		50%	3	5	1	11	0	1
● Utf8(String, Int)		0%		n/a	1	1	3	3	1	1
● getBytes()		0%		n/a	1	1	1	1	1	1
● toString()		90%		75%	1	3	1	5	0	1
● setByteLength(Int)		100%		100%	0	2	0	7	0	1
● Utf8(String)		100%		n/a	0	1	0	8	0	1
● Utf8(Utf8)		100%		n/a	0	1	0	6	0	1
● Utf8(byte[])		100%		n/a	0	1	0	6	0	1
● compareTo(Utf8)		100%		n/a	0	1	0	1	0	1
● writeExternal(ObjectOutput)		100%		n/a	0	1	0	3	0	1
● readExternal(ObjectInput)		100%		n/a	0	1	0	3	0	1
● Utf8()		100%		n/a	0	1	0	4	0	1
● subSequence(Int, Int)		100%		n/a	0	1	0	1	0	1
● charAt(Int)		100%		n/a	0	1	0	1	0	1
● length()		100%		n/a	0	1	0	1	0	1
● getBytesFor(String)		100%		n/a	0	1	0	1	0	1
● static {...}		100%		n/a	0	1	0	1	0	1
● getByteLength()		100%		n/a	0	1	0	1	0	1
Total	154 of 406	62%	25 of 44	43%	20	44	35	105	5	22

Figure 2: Report JaCoCo della suite manuale su Utf8.

Resolver

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
● unionEquiv(Schema, Schema, Map)		0%		0%	23	23	36	36	1	1
● resolve(Schema, Schema, GenericData, Map)		0%		0%	13	13	22	22	1	1
● resolve(Schema, Schema, GenericData)		0%		n/a	1	1	1	1	1	1
● resolve(Schema, Schema)		0%		n/a	1	1	1	1	1	1
● Resolver()		100%		n/a	0	1	0	1	0	1
Total	339 of 342	0%	57 of 57	0%	38	39	60	61	4	5

Figure 3: Report JaCoCo della suite Randoop isolata su Resolver.

Prompt 1 - Zero-shot senza codice sorgente:

Scrivi test JUnit 5 per il metodo statico `Resolver.resolve(Schema writer, Schema reader)` della libreria Apache Avro (package `org.apache.avro`). Non fornirò il codice sorgente: basati solo sulla tua conoscenza di Avro e della sua logica di schema resolution.

Prompt 2 - Zero-shot con codice sorgente:

Scrivi test JUnit 5 per la seguente classe Java. Copri tutti i casi che ritieni rilevanti.

[codice sorgente completo di `Resolver.java`]

Prompt 3 - Con vincoli di progetto espliciti:

Scrivi test JUnit 5 per la classe `Resolver` di Apache Avro (codice sotto). Segui queste regole:

- package `org.apache.avro`
- niente wildcard import, solo import espliciti
- indentazione a 2 spazi (il progetto usa Spotless con questa configurazione)
- NON usare `@Nested`: la configurazione Surefire di questo progetto esclude le classi interne (pattern `**/*$*`), quindi tutti i metodi `@Test` devono stare in un'unica classe a livello singolo
- copri sistematicamente queste categorie di risoluzione schema writer/reader: tipi primitivi uguali (`DoNothing`), promozioni numeriche valide (`Promote`), combinazioni incompatibili (`ErrorAction`), `FIXED` (nome/size uguali o diversi), `ARRAY/MAP` (`Container`), `ENUM` (simboli uguali o nome diverso), `RECORD` (campi aggiunti/rimossi, con o senza default), `UNION` (solo lato reader, solo lato writer)
- per ogni test aggiungi un commento breve su quale categoria copre

[codice sorgente completo di `Resolver.java`]

Table 9: Test completo dei tre prompt utilizzati per la generazione LLM su Resolver.

Resolver

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods
unionEquiv(Schema.Schema.Map)		68%		57%	15 23	13 36	0 1
resolve(Schema.Schema.GenericData.Map)		94%		89%	2 13	1 22	0 1
Resolver()		0%	n/a	n/a	1 1	1 1	1 1
resolve(Schema.Schema.GenericData)		100%	n/a	n/a	0 1	0 1	0 1
resolve(Schema.Schema)		100%	n/a	n/a	0 1	0 1	0 1
Total	71 of 342	79%	18 of 57	68%	18 39	15 61	1 5

Figure 4: Report JaCoCo della suite LLM (Prompt 2+3) isolata su Resolver.

Prompt	Test generati	Esito	Interventi manuali
1 (senza codice)	—	Non compilabile	N/A
2 (con codice)	45	45/45 passanti	Appiattimento @Nested
3 (con vincoli)	13	13/13 passanti	Nessuno

Table 10: Risultati dei tre prompt LLM (Gemini) su Resolver.

Metrica	Valore
Test generati	78
Statement coverage (JaCoCo)	88.5%
Branch coverage (JaCoCo)	84.2%

Table 11: Risultati della generazione EvoSuite su Resolver (budget 60s), copertura misurata con JaCoCo sulla suite isolata.

Resolver

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods
unionEquiv(Schema.Schema.Map)		74%		78%	6 23	6 36	0 1
resolve(Schema.Schema.GenericData.Map)		94%		94%	1 13	1 22	0 1
resolve(Schema.Schema.GenericData)		100%	n/a	n/a	0 1	0 1	0 1
resolve(Schema.Schema)		100%	n/a	n/a	0 1	0 1	0 1
Resolver()		100%	n/a	n/a	0 1	0 1	0 1
Total	56 of 342	83%	9 of 57	84%	7 39	7 61	0 5

Figure 5: Report JaCoCo della suite EvoSuite isolata su Resolver.

Utf8

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods
hashCode()		0%		0%	5 5	11 11	1 1
writeExternal(ObjectOutput)		0%	n/a	n/a	1 1	3 3	1 1
readExternal(ObjectInput)		0%	n/a	n/a	1 1	3 3	1 1
Utf8(String.int)		0%	n/a	n/a	1 1	3 3	1 1
compareSequences(CharSequence.CharSequence)		96%		87%	2 9	1 12	0 1
equals(Object)		96%		91%	1 7	1 14	0 1
set(Utf8)		100%		100%	0 2	0 7	0 1
setByteLength(int)		100%		100%	0 2	0 7	0 1
set(String)		100%	n/a	n/a	0 1	0 8	0 1
Utf8(String)		100%	n/a	n/a	0 1	0 8	0 1
Utf8(Utf8)		100%	n/a	n/a	0 1	0 6	0 1
toString()		100%		100%	0 3	0 5	0 1
Utf8(byte[])		100%	n/a	n/a	0 1	0 6	0 1
compareTo(Utf8)		100%	n/a	n/a	0 1	0 1	0 1
Utf8()		100%	n/a	n/a	0 1	0 4	0 1
subSequence(int.int)		100%	n/a	n/a	0 1	0 1	0 1
charAt(int)		100%	n/a	n/a	0 1	0 1	0 1
length()		100%	n/a	n/a	0 1	0 1	0 1
getBytesFor(String)		100%	n/a	n/a	0 1	0 1	0 1
static {...}		100%	n/a	n/a	0 1	0 1	0 1
getBytes()		100%	n/a	n/a	0 1	0 1	0 1
getByteLength()		100%	n/a	n/a	0 1	0 1	0 1
Total	75 of 406	81%	11 of 44	75%	11 44	22 105	4 22

Figure 6: Report JaCoCo della suite Randoop isolata su Utf8.

Utf8

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods
• Utf8(String, int)		0%		n/a	1 1	3 3	1 1
• compareSequences(CharSequence, CharSequence)		100%		93%	1 9	0 12	0 1
• equals(Object)		100%		100%	0 7	0 14	0 1
• hashCode()		100%		87%	1 5	0 11	0 1
• set(Utf8)		100%		100%	0 2	0 7	0 1
• setByteLength(int)		100%		100%	0 2	0 7	0 1
• set(String)		100%		n/a	0 1	0 8	0 1
• Utf8(String)		100%		n/a	0 1	0 8	0 1
• Utf8(Utf8)		100%		n/a	0 1	0 6	0 1
• toString()		100%		100%	0 3	0 5	0 1
• Utf8(byte[])		100%		n/a	0 1	0 6	0 1
• compareTo(Utf8)		100%		n/a	0 1	0 1	0 1
• writeExternal(ObjectOutput)		100%		n/a	0 1	0 3	0 1
• readExternal(ObjectInput)		100%		n/a	0 1	0 3	0 1
• Utf8()		100%		n/a	0 1	0 4	0 1
• subSequence(int, int)		100%		n/a	0 1	0 1	0 1
• charAt(int)		100%		n/a	0 1	0 1	0 1
• length()		100%		n/a	0 1	0 1	0 1
• getBytesFor(String)		100%		n/a	0 1	0 1	0 1
• static {...}		100%		n/a	0 1	0 1	0 1
• getBytes()		100%		n/a	0 1	0 1	0 1
• getByteLength()		100%		n/a	0 1	0 1	0 1
Total	7 of 406	98%	2 of 44	95%	3 44	3 105	1 22

Figure 7: Report JaCoCo della suite LLM (Prompt 2+3) isolata su Utf8.

Prompt 1 - Zero-shot senza codice sorgente:

Scrivi test JUnit 5 per la classe Utf8 della libreria Apache Avro (package org.apache.avro.util). La classe rappresenta una sequenza di byte codificata UTF-8, con metodi per impostare/ottenere lunghezza e contenuto in byte, confronto e conversione a String. Non fornirò il codice sorgente: basati solo sulla tua conoscenza di Avro e del formato UTF-8.

Prompt 2 - Zero-shot con codice sorgente:

Scrivi test JUnit 5 per la seguente classe Java. Copri tutti i casi che ritieni rilevanti.

[codice sorgente completo di Utf8.java]

Prompt 3 - Con vincoli di progetto espliciti:

Scrivi test JUnit 5 per la classe Utf8 di Apache Avro (codice sotto). Segui queste regole:

- package org.apache.avro.util
- niente wildcard import, solo import espliciti
- indentazione a 2 spazi (il progetto usa Spotless con questa configurazione)
- NON usare @Nested: la configurazione Surefire di questo progetto esclude le classi interne (pattern **/*\$*), quindi tutti i metodi @Test devono stare in un'unica classe a livello singolo
- copri sistematicamente: costruttori (vuoto, da String, da byte[], da altro Utf8), getBytes/setBytes, getByteLength/setByteLength (inclusa l'espansione E la riduzione dell'array interno), equals/hashCode, compareTo, toString, casi limite come stringhe vuote o con caratteri multi-byte UTF-8
- per ogni test aggiungi un commento breve su cosa copre

[codice sorgente completo di Utf8.java]

Table 12: Testi completi dei tre prompt utilizzati per la generazione LLM su Utf8.

Prompt	Test generati	Esito	Interventi manuali
1 (senza codice)	—	Nessuna allucinazione evidente	N/A (non integrato)
2 (con codice)	23	23/23 passanti	Appiattimento @Nested
3 (con vincoli)	17	16/16 passanti	Rimozione di 1 test (costruttore package-private)

Table 13: Risultati dei tre prompt LLM (Gemini) su Utf8.

Metrica	Valore
Test generati	63
Statement coverage (JaCoCo)	95.2%
Branch coverage (JaCoCo)	95.5%

Table 14: Risultati della generazione EvoSuite su Utf8 (budget 60s), copertura misurata con JaCoCo sulla suite isolata.

Utf8

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods
readExternal(ObjectInput)		0%	n/a		1 1	3 3	1 1
compareSequences(CharSequence, CharSequence)		91%		93%	1 9	1 12	0 1
equals(Object)		96%		91%	1 7	1 14	0 1
hashCode()		100%		100%	0 5	0 11	0 1
set(Utf8)		100%		100%	0 2	0 7	0 1
setByteLength(int)		100%		100%	0 2	0 7	0 1
set(String)		100%	n/a		0 1	0 8	0 1
Utf8(String)		100%	n/a		0 1	0 8	0 1
Utf8(Utf8)		100%	n/a		0 1	0 6	0 1
toString()		100%		100%	0 3	0 5	0 1
Utf8(byte[])		100%	n/a		0 1	0 6	0 1
compareTo(Utf8)		100%	n/a		0 1	0 1	0 1
writeExternal(ObjectOutput)		100%	n/a		0 1	0 3	0 1
Utf8()		100%	n/a		0 1	0 4	0 1
Utf8(String, int)		100%	n/a		0 1	0 3	0 1
subSequence(int, int)		100%	n/a		0 1	0 1	0 1
charAt(int)		100%	n/a		0 1	0 1	0 1
length()		100%	n/a		0 1	0 1	0 1
getBytesFor(String)		100%	n/a		0 1	0 1	0 1
static (...)		100%	n/a		0 1	0 1	0 1
getBytes()		100%	n/a		0 1	0 1	0 1
getByteLength()		100%	n/a		0 1	0 1	0 1
Total	17 of 406	95%	2 of 44	95%	3 44	5 105	1 22

Figure 8: Report JaCoCo della suite EvoSuite isolata su Utf8.

Famiglia	Classe	Statement Coverage	Branch Coverage
BB	Resolver	42.6% (26/61)	31.6% (18/57)
BB	Utf8	66.7% (70/105)	43.2% (19/44)
RND	Resolver	1.6% (1/61)	0.0% (0/57)
RND	Utf8	79.0% (83/105)	75.0% (33/44)
LLM	Resolver	75.4% (46/61)	68.4% (39/57)
LLM	Utf8	97.1% (102/105)	95.5% (42/44)
ES	Resolver	88.5% (54/61)	84.2% (48/57)
ES	Utf8	95.2% (100/105)	95.5% (42/44)

Table 15: Confronto di adeguatezza (Statement/Branch coverage) tra le quattro famiglie di test, per ciascuna classe.

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
2	60% 100/166	49% 54/110	92% 54/59

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
org.apache.avro	1	43% 26/61	41% 21/51	100% 21/21
org.apache.avro.util	1	70% 74/105	56% 33/59	87% 33/38

Figure 9: Report PIT (indice generale), dopo l'integrazione dei test MT su Utf8.

Classe	Line Coverage	Mutation Coverage	Test Strength
Resolver (suite BB)	43% (26/61)	41% (21/51)	100% (21/21)
Utf8 (suite BB)	67% (70/105)	37% (22/59)	65% (22/34)
Utf8 (suite BB + MT)	70% (74/105)	56% (33/59)	87% (33/38)

Table 16: Risultati del mutation testing con PIT, prima e dopo l'integrazione dei test MT su Utf8.

Variante	BB	RND (campione)	LLM	ES
C1	FAIL (compilazione)	FAIL (stesso motivo)	FAIL (stesso motivo)	PASS 78/78
C2	PASS 32/32	PASS 500/500	PASS 45+13/45+13	PASS 78/78
C3	PASS 32/32	PASS 500/500	PASS 45+13/45+13	PASS 78/78
C4	PASS 32/32	PASS 500/500	PASS 45+13/45+13	PASS 78/78

Table 17: Validazione delle famiglie di test su ciascuna variante di `Resolver`.

Metrica	C0	C1	C2	C3	C4
Statement Coverage	92.3%	84.7% ¹	93.1%	92.2%	92.2%
Branch Coverage	83.6%	68.1% ¹	83.8%	82.8%	82.8%
Mutation Score	51%	N/A ²	51%	51%	51%
LOC (proxy smell)	505	457	457	485	485
Punti di decisione	191	163	163	179	179

Table 18: Delta di coverage, mutation score e proxy di smell tra C0 e le varianti C1-C4 di `Resolver`. ¹C1 misurato solo su ES (unica famiglia compilante), non comparabile 1:1. ²Non misurabile: BB/RND/LLM non compilano contro C1, ES incompatibile con PIT.

Famiglia	C1	C2	C3	C4
BB (35 test)	PASS 35/35	PASS 35/35	PASS 35/35	PASS 35/35
MT (14 test)	FAIL 13/14	FAIL 13/14	PASS 14/14	PASS 14/14
RND (campione 500)	FAIL (compilazione)	PASS 500/500	PASS 500/500	PASS 500/500
LLM (Prompt 2+3)	FAIL (compilazione)	PASS 23+17/23+17	PASS 23+17/23+17	PASS 23+17/23+17
ES (63 test)	FAIL (compilazione)	FAIL 61/63	PASS 63/63	FAIL 62/63

Table 19: Validazione delle cinque famiglie di test su ciascuna variante di `Utf8`.

Metrica	C0	C1	C2	C3	C4
Statement Coverage	100%	68.7% ¹	100%	100%	100%
Branch Coverage	97.7%	47.2% ¹	97.2%	97.7%	97.5%
Mutation Score	86%	N/A ²	N/A ²	86%	85%
LOC (proxy smell)	178	172	172	177	175
Punti di decisione	27	24	24	27	25

Table 20: Delta di coverage, mutation score e proxy di smell tra C0 e le varianti C1-C4 di `Utf8`. ¹Misurato solo su BB+MT (2 famiglie su 5). ²PIT rifiuta l'esecuzione: la suite MT non è verde su C1/C2.

Famiglia	Naming dei metodi di test	Evidenza osservata nel codice generato
BB / MT	Descrittivo, per categoria (es. <code>testSetByteLengthAumento</code>)	Scritti manualmente; commento di intento su ogni test
LLM (Prompt 2-3)	Descrittivo (es. <code>testSetByteLengthExpansion</code> , <code>testCopyConstructor</code>)	@DisplayName in linguaggio naturale; commento di categoria per test (Prompt 3)
Randoop	Sequenziale (<code>test0001</code> , <code>test0002</code> , ...)	Commenti solo automatici ("The following exception was thrown during execution..."), non di intento
EvoSuite	Sequenziale (<code>test00</code> , <code>test01</code> , ...)	Nessun commento descrittivo; input spesso stringhe casuali prive di significato (es. <code>"-5.6D8[[]/?/nW,u#G"</code>)

Table 21: Caratterizzazione qualitativa della leggibilità dei test prodotti da ciascuna famiglia, basata su ispezione diretta del codice sorgente generato in entrambe le classi.